An introduction to Python language

Lecture 1

Constructor

Dr. Ahmed H. Aliwy

University of Kufa

2020

This lecture are constructed from the following lectures:

1. Python help
2. http://www.tutorialspoint.com/
3. Python Programming: An Introduction to Computer Science by John Zelle
4. An Introduction to Python by Dr. Nancy Warter-Perez
5. Introduction to Programming with Python by Marty Stepp
6. Introduction to Python by Chen lin
7. Some others …

Python history

- Python was conceived in the late 1980s
- its implementation was started in December 1989 by Guido van Rossum.
- In February 1991, Van Rossum published the code (labeled version 0.9.0) to alt.sources.
- Python 1.0 –1994.
- Python 2.0 – 2000.
- Python 3.0 –2008.
  - Python 3.9 – 2020.

Python Characteristics

- **Python is Interpreted:** This means that it is processed at runtime by the interpreter and you do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** This means that you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** This means that Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is Beginner's Language:** Python is a great language for the beginner programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Python Characteristics

- **Easy-to-learn**: Python has relatively :
  - few keywords,
  - simple structure,
  - a clearly defined syntax.
  - This allows the student to pick up the language in a relatively short period of time.
- **Easy-to-read:**
  - Python code is much more clearly defined and visible to the eyes.
- **Easy-to-maintain:**
  - Python's success is that its source code is fairly easy-to-maintain.
- **A broad standard library:**

Python Characteristics

- **Portable:**
  - Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:**
  - You can add low-level modules to the Python interpreter. **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:**
  - Python supports GUI applications that can be created and ported to many system calls
- **Scalable:**
  - Python provides a better structure and support for large programs than shell scripting.

Python Characteristics

- Support for functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- Very high-level dynamic data types and supports dynamic type checking.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA and Java.

Major Versions of Python

- "Python" or "CPython" is written in C/C++
  - Version 2.7 came out in mid-2010
  - Version 3.1.2 came out in early 2010
- "Jython" is written in Java for the JVM
- "IronPython" is written in C# for the .Net environment

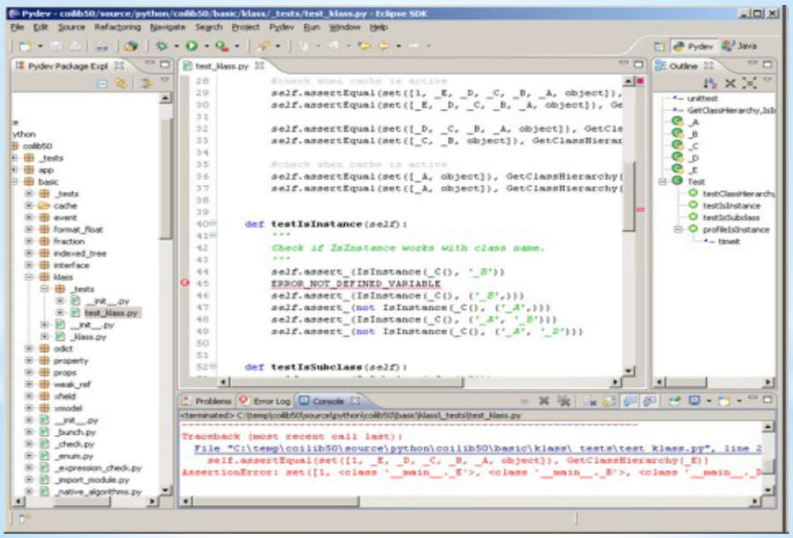
Development Environments

1. PyDev with Eclipse
2. Komodo
3. Emacs
4. Vim
5. TextMate
6. Gedit
7. **Idle**
8. PIDA (Linux)(VIM Based)
9. **NotePad++ (Windows)**
10. BlueFish (Linux)

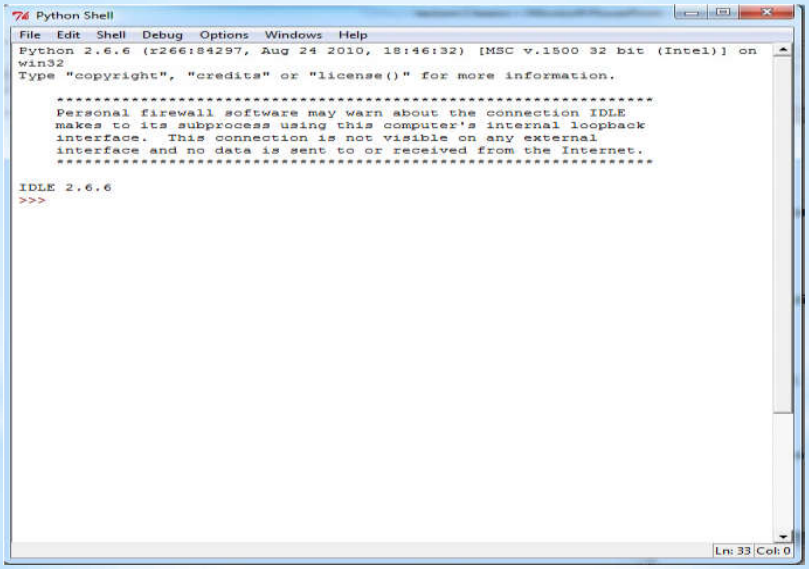
Downloading

1. Open a Web browser and go to http://www.python.org/download/
2. Follow the link for the Windows installer *python-XYZ.msi* file where XYZ is the version you are going to install.
3. To use this installer *python-XYZ.msi*, the Windows system must support Microsoft Installer 2.0. Just save the installer file to your local machine and then run it to find out if your machine supports MSI.
4. Run the downloaded file by double-clicking it in Windows Explorer. This brings up the Python install wizard, which is really easy to use. Just accept the default settings, wait until the install is finished, and you're ready to roll!

Interactive python



Interactive python



Python functions and methods

- **Built-in functions**: The Python interpreter has a number of built-in functions to it that are always available.
- **None Built-in functions (in Module):** The Python interpreter also has other instructions which are not available at any time but we can call them by importing the container module.
  - Any developer can build his own module with private function

Built-in functions

Built-in Functions				
abs()	divmod()	input()	open()	staticmethod()
all()	enumerate()	int()	ord()	str()
any()	eval()	isinstance()	pow()	sum()
basestring()	execfile()	issubclass()	print()	super()
bin()	file()	iter()	property()	tuple()
bool()	filter()	len()	range()	type()
bytearray()	float()	list()	raw_input()	unichr()
callable()	format()	locals()	reduce()	unicode()
chr()	frozenset()	long()	reload()	vars()
classmethod()	getattr()	map()	repr()	xrange()
cmp()	globals()	max()	reversed()	zip()
compile()	hasattr()	memoryview()	round()	__import__()
complex()	hash()	min()	set()	apply()
delattr()	help()	next()	setattr()	buffer()
dict()	hex()	object()	slice()	coerce()
dir()	id()	oct()	sorted()	intern()

Some examples

```
>>> abs(-5)
        5
>>> int (2.5)
        2
>>> print "ali"
        ali
>>> pow(2,3)
        8
>>> pow(2,3,3) # more efficient than (2**3)%3
        2
>>> pow(5,3,3)
        2
```

None built-in functions (in Module)

- Firstly we import the specific module then we can use these functions.
- We can use simple cases for importing:
  - Import module-Name
  - from module-Name import *
  - from module-Name import function

math module

```
>>> import math
>>> math.cos(50)
0.96496602849211333
>>>
```

Command name	Description
`abs(`***value***`)`	absolute value
`ceil(`***value***`)`	rounds up
`cos(`***value***`)`	cosine, in radians
`floor(`***value***`)`	rounds down
`log(`***value***`)`	logarithm, base *e*
`log10(`***value***`)`	logarithm, base 10
`max(`***value1***`, `***value2***`)`	larger of two values
`min(`***value1***`, `***value2***`)`	smaller of two values
`round(`***value***`)`	nearest whole number
`sin(`***value***`)`	sine, in radians
`sqrt(`***value***`)`	square root

...

math module

- There are other function in this module
- Also, We have constants:

```
>>> math.e
2.7182818284590451
>>> math.pi
3.1415926535897931
>>>
```

Different importing examples

```
>>> import math
>>> math.cos(50)
0.96496602849211333
>>> math.e
2.7182818284590451
>>> math.pi
3.1415926535897931
>>> from math import cos
>>> cos(50)
0.96496602849211333
>>> sin(50)

Traceback (most recent call last):
  File "<pyshell#47>", line 1, in <module>
    sin(50)
NameError: name 'sin' is not defined
>>> from math import *
>>> sin(50)
-0.26237485370392877
>>>
```

Error

Variables

- **variable**: A named piece of memory that can store a value.
- **assignment statement**: Stores a value into a variable.
  - Syntax:

    name = ***value***
- Variable does not need to type definition.

```
>>> x=10;z=20;y=30
>>> p=x+y
>>> p
40
>>> f=b=g=100
>>> f
100
>>>
```

Variables

- **Names of variable:** In Python, the names of the variables can be placed under the following conditions:
  - The variable name can consist of letters and numbers.
  - The variable name always begins with a letter.
  - The underscore (_) can be placed anywhere in the variable names, even if it is at the beginning of the variable name.
  - The variable name must not be a keyword (a word used by Python) or the name of a unit that was imported into the program

- **Examples**

Name	Case
2Ali	No
ali	Yes
Ali	Yes
_Ali	Yes
A12a	Yes
A_	Yes
A_li	Yes
if	No (reserved word)
elif	No (reserved word)
...	

Scope of Variables

- Scope of variables can be
  - Local variables:
    - Variables that are defined inside a function body have a local scope
    - local variables can be accessed only inside the function in which they are declared
  - Global variables
    - Variables that are defined outside have a global scope.
    - global variables can be accessed throughout the program body by all functions

Operators

بايثون كغيرها من اللغات تستخدم العديد من المعاملات ويمكن تقسيمها الى:

- معاملات رياضية تستعمل في العمليات الحسابية
- معاملات مقارنة تستعمل في العمليات الشرطية
- معاملات اسناد وهي المساواة (=).
- معاملات تعمل على البتات
- معاملات فحص العضوية والتماثل
- معاملات المنطق وتستخدم عادة لربط معاملات المقارنة

Operators

العمليات الرياضيه

المعامل	العملية	مثال في البايثون a=10; b=5	النتيجة
+	اضافة (جمع)	Y=a+b	15
-	نقصان (طرح)	Y= a-b	10
*	ضرب	Y=a*b	50
/	قسمة	Y=a/b	2
%	باقي القسمة	Y=a%b	0
**	اس	Y=a**b	100000
//	قسمة مقربة (قسمة صحيحة)	Y=a//b	2

Operators

معاملات المقارنة

المعامل	الدلالة	الملاحظات
==	يساوي	اختبار تساوي متغيرين او متغير و قيمة
=!	لايساوي	اختبار عدم تساوي متغيرين او متغير و قيمة
>	اكبر من	اختبار متغير اكبر من متغير او قيمة
<	اصغر من	اختبار متغير اصغر من متغير او قيمة
>=	اكبر او يساوي	اختبار متغير اكبراو يساوي متغير او قيمة
<=	اصغر او يساوي	اختبار متغير اصغراو يساوي متغير او قيمة

Operators

معاملات البتات

المعامل	الدلالة
|	Or (او)
^	Exclusive or (او محددة)
&	And (و)
<<	Shift left (تزحيف الى اليسار)
>>	Shift right (تزحيف الى اليمين)
~	not (نفي)

Operators

معاملات فحص العضوية والتماثل

المعامل	الدلالة
In	تفحص متغير او قيمة ضمن مجموعة قيم (قائمة مثلا)
not in	فحص بعدم الوجود
is	فحص هل يوجد تطابق
is not	فحص بعدم وجود تطابق

معاملات المنطق

المعامل	الدلالة
or ||	يستخدم لربط شرطين يتحقق احدهما فيعطي نتيجة true
and &&	يستخدم لربط شرطين يتحقق كلاهما فيعطي نتيجة true
not	يستخدم لتدقيق عدم الوجود

Exercise

compute the roots of a quadratic equation:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Python basics

- Numbers
- String
- Lists
- Tuples
- Dictionaries
- …

Numbers

- The default Number system is the decimal but there are other as Binary, Octal and hexadecimal numbers
  Ex: 0o177, 0x9ff, 0xff, 0b01010101
- Integers –represent whole numbers (unlimited size)
  Ex: `3, -7, 123, 76`
  Ex: `99999999999999999999999999999999999999`
- Floating-point – represent numbers with decimal places
  Ex: `1.2, 3.14159,3.14e-10`
- Complex numbers
  Ex: 3+4j, 3.0+4.0j, 3J

Converting among number types

- We can convert from one type to other using:
  - int(x), to integer
  - float(x), to float
  - complex(x,y), to complex

```
>>> int(10-5)
5

>>> float(15)
15.0

>>> complex(7)
(7+0j)

>>> complex(7,6)
 (7+6j)
```

Numbers: Little Notes

- **precedence**: Order in which operations are computed.
  - `* / % **` have a higher precedence than `+ -`

    `1 + 3 * 4` is `13`

  - Parentheses can be used to force a certain order of evaluation.

    `(1 + 3) * 4` is `16`

Strings

- Text is represented in programs by the *string* data type.
- A string is a sequence of characters enclosed within quotation marks (") or apostrophes (').

```
>>> str1="Hello"
>>> str2='spam'
>>> print(str1, str2)
Hello spam
>>> type(str1)
<class 'str'>
>>> type(str2)
<class 'str'>
```

Strings

- **Getting a string as input**

```
>>> firstName = input("Please enter your name: "
Please enter your name: "ahmed Hussein"
>>> print("Hello", firstName)
('Hello', 'ahmed Hussein')
```

- **Get multiple input:**

```
>>> yourName, age=input("write your name and your age separated by space: ").split()
write your name and your age separated by space: Ahmed 46
>>> yourName
'Ahmed'
>>> age
'46'
>>>
```

- **Getting a value as input**

```
>>> age = eval(input("Please enter your age: "))
Please enter your age: "40"
>>> print age
40
>>>
```

Strings

- Getting a character from string.

```
>>> greet = "Hello Bob"
>>> greet[0]
'H'
>>> print(greet[0], greet[2], greet[4])
H l o
>>> x = 8
>>> print(greet[x - 2])
B
```

Lists Data structure

- Strings are always sequences of characters, but *lists* can be sequences of arbitrary values.
- Lists can have numbers, strings, or both!

  myList = [1, "Spam ", 4, "U"]
- Lists are *mutable*, meaning they can be changed. Strings can **not** be changed.
  - myList = [34, 26, 15, 10]
  - mylist[2]=5 => myList is [34, 26, 5, 10]

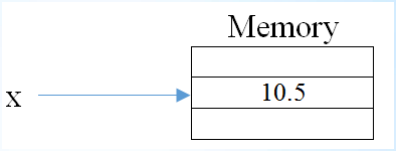
Lists Data structure

mylist=["a","b",3.58,"d",4,0]		
mylist[0] mylist[2]	a 3.58	Indexing
mylist[-1] mylist[-2]	0 4	Negative indexing (counts from end)
mylist[1:4]	["b",3.58,"d"]	Slicing (like strings)
"b" in mylist "e" not in mylist	1 or True 1 or True	
mylist.append(8)	["a","b",3.58,"d",4,0,8]	Add to end of list

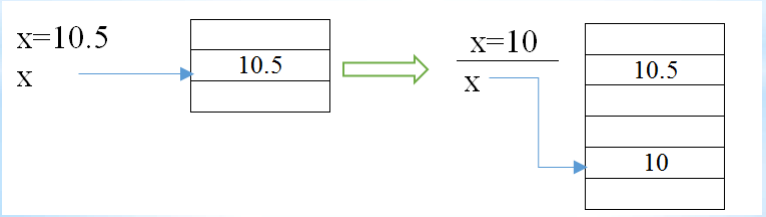
Mutable and Immutable Data Type

Immutable Types: numbers, strings, Tuples

```
>>> x=10.5
```



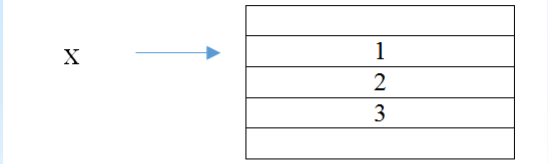

```
>>> x=10
```



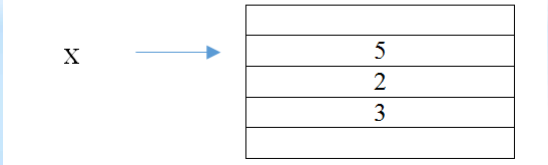
Mutable and Immutable Data Type

Mutable Types: Lists, dictionaries, user defined types

```
>>> x=[1,2,3]
```




```
>>> x[0]=5
```



Mutable and Immutable Data Type

Examples:

Immutable	mutable
المثال الاول	المثال الثاني

`>>> a=3.5`
`>>> b=a`
`>>> b=2`
`>>> print(a)`
`3.5`
	`>>> a=[1,2,3]`
	`>>> b=a`
	`>>> b[1]=5`
	`>>> print(a)`
	`[1,5,3]`

Interactive mode and Script Mode

Interactive mode

```
>>> x=10

>>> y=x+1
>>> y
 11
```

Script mode

test.py

File➔new file ➔

```
x=10

y=x+1
print(y)
```

File➔Save as➔ test.py

For running test.py : Run➔ Run Module